

Faculty of Information Engineering, Informatics, and Statistics
Master Degree in Engineering in Computer Science

N. Sane Runner

Interactive Graphics Project

Giovanni Canedoli 1991710

Professor:
Marco Schaerf

A.Y. 2025-2026

Contents

1	Introduction	3
1.1	Project Idea	3
1.2	Libraries Used	3
1.2.1	THREE.js	3
1.2.2	Tween.js	3
2	Libraries, Tools, and External Assets	4
3	Game Environment	4
3.1	Menus	4
3.2	Maps	4
3.3	Playable Characters	5
3.3.1	Crash Bandicoot	5
3.3.2	Dr. Neo Cortex	5
3.4	Hazards and Collectables	5
3.4.1	Boxes	5
3.4.2	Wumpa Fruits	5
3.4.3	Gems	5
3.4.4	Gears	5
4	Gameplay	6
4.1	Controls	6
4.2	Recommended Play Flow	6
5	Rendering Pipeline	6
5.1	Scene Setup	6
5.2	Textures and Materials	6
5.3	Lighting	7
5.4	Camera	7
6	Technical Aspects	7
6.1	Asset Loading	7
6.2	Main Game Loop	7
6.3	Map Generation	7
6.4	Object Pooling and Memory Management	8
6.5	Collision Detection	8
6.6	Pause System	8
6.7	Settings System	8
7	Animations	9
7.1	Character Animations	9
7.1.1	Forward Movement	9
7.1.2	Run	9
7.1.3	Jump Animation	9
7.1.4	Lateral Slide Animation	9
7.1.5	Crash Spin Attack	9
7.1.6	Cortex Punch Attack	10
7.2	Object Animations	10
7.2.1	Wumpa Fruits	10
7.2.2	Nitro Boxes	10
7.2.3	Dropped Items	10

7.2.4	Aku Aku Startup	10
7.2.5	Gears (Hazards)	10
8	Audio System	11
9	File Structure and Organisation	11
10	Conclusions	12

1 Introduction

1.1 Project Idea

N. Sane Runner is a browser-based 3D endless runner game inspired by the classic Crash Bandicoot franchise. The player controls either Crash Bandicoot or Dr. Neo Cortex as they auto-run through procedurally generated terrain divided into three lanes, collecting items, smashing crates, and dodging hazards for as long as possible.

The project was developed as a final assignment for the Interactive Graphics course at Sapienza University of Rome. It represents an opportunity to apply three-dimensional rendering, procedural generation, skeletal animation, and real-time collision detection in a complete, playable product delivered entirely in the browser.

1.2 Libraries Used

1.2.1 *THREE.js*

THREE.js is the main rendering library employed throughout the project. It is a lightweight, cross-browser JavaScript library built on top of WebGL that provides high-level abstractions for managing a 3D scene, including cameras, lights, shadows, meshes, geometries, materials, and textures. All rendering in N. Sane Runner is driven by THREE.js.

1.2.2 *Tween.js*

Tween.js is a JavaScript tweening engine used to create smooth programmatic animations. It drives all character and object animations in the project—including walk cycles, jump arcs, attack motions, and item dropping. *No pre-existing animations* are used; every animation is defined procedurally in JavaScript.

Source code is divided into focused modules, each with a clearly defined responsibility (see Section 9), reducing coupling and keeping individual files manageable. Version control was handled with **Git**, hosted on a private GitHub repository under the SapienzaInteractiveGraphicsCourse organisation.

2 Libraries, Tools, and External Assets

Component	Role in the project
THREE.js	3D rendering, scene graph, meshes, materials, cameras, lights, and textures.
Tween.js	Procedural character and object animations via keyframe tweening.
GLTFLoader	Import of external GLTF/GLB 3D models.
FBXLoader	Import of FBX models (used for Dr. Neo Cortex).
Vite	Build tool, dev server, and GitHub Pages deployment pipeline.
HTML5 Audio	Music playback, ambient sound, and interaction sound effects.

Table 1: Main libraries and tools used in the project.

All 3D models are stored in the `public/` directory and loaded at startup. The models include Crash Bandicoot, Dr. Neo Cortex, and all the other assets. The Crash Bandicoot original soundtrack and sound effects are similarly loaded from `public/`.

3 Game Environment

3.1 Menus

The game features four main screens:

- **Main menu (`index.html`):** The landing page, shown after all assets finish loading. The player can select the playable character, choose a difficulty, choose a map, and start the game.
- **Pause menu (`pause.html`):** Accessible at any time during a run by pressing `Escape`. It allows the player to toggle audio, resume, or quit to the main menu.
- **Game Over screen (`gameover.html`):** Displayed when the player runs out of lives.
- **HUD (`hud.html`):** An HTML overlay rendered on top of the 3D canvas during gameplay, showing score, lives, distance, and the collected gem count in real time.

3.2 Maps

Three distinct environments are available, each with its own tile textures, and side-road objects:

- **N. Sanity Beach**
- **Jungle Rollers**
- **Cortex Power**

Each map is divided into two regions: the **main road**, a three-lane strip where all gameplay occurs, and the **sides of the road**, which contain non-interactive decorative objects that reinforce the chosen environment.

3.3 Playable Characters

3.3.1 *Crash Bandicoot*

Crash is the default playable character, loaded from a GLTF/GLB model. He is a hierarchical skeleton model whose joints are individually referenced and manipulated at runtime to produce all in-game animations (running, jumping, sliding, attacking, idle). The rigged model, downloaded from sketchfab, consists of 53 bones.

3.3.2 *Dr. Neo Cortex*

Dr. Neo Cortex is the second playable character, loaded from an FBX model. He shares the same animation system as Crash, with bone references remapped to his specific skeleton. Selecting Cortex changes the character's silhouette, proportions, and accompanying audio cues. The model was downloaded from sketchfab. Initially the model wasn't rigged. It was rigged further using blender. The armature consists of 65 bones.

None of the two characters had pre downloaded animations.

3.4 Hazards and Collectables

3.4.1 *Boxes*

Boxes are the primary interactive objects placed on the main road. Five variants exist:

- **Standard (Wooden) Box:** Drops Wumpas and awards points. Inflicts damage if collided without triggering a spin or punch attack.
- **Nitro Box:** Explodes on contact, dealing damage to the player.
- **Aku Aku Box:** Drops Aku Aku when attacked; harmful if touched from the side or top.
- **Question Box:** Drops a random reward (either a gem, a new life or a number between 3 to 5 of wumpas).
- **New Life Box:** Grants an extra life when collected.

3.4.2 *Wumpa Fruits*

Wumpa fruits are small collectables arranged in lines across the lanes. Collecting one hundred fruits awards one bonus life, consistent with the original Crash Bandicoot games.

3.4.3 *Gems*

Five coloured gems (blue, green, purple, red, yellow) are hidden throughout runs. The player can connect the 5 gems in the run.

3.4.4 *Gears*

Gears are rotating hazards placed on the road. Contact with a gear deals damage; they cannot be smashed and must be avoided.

4 Gameplay

The objective of the game is to run as far as possible while collecting Wumpa fruits and gems, smashing boxes, and avoiding hazards until all lives are lost.

4.1 Controls

Input	Action
A / Arrow Left	Move left one lane
D / Arrow Right	Move right one lane
W / Arrow Up / Space	Jump
Mouse left click	Attack
Escape	Pause / resume

Table 2: Player controls.

4.2 Recommended Play Flow

1. On the main menu, select a character, difficulty, and map, then press **Start Game**.
2. During the first few seconds the player is given a clear section of road to get used to the controls.
3. Lane-switch left and right to avoid gears and Nitro boxes.
4. Jump over low obstacles and low-placed gears.
5. Attack standard, Question, and New Life boxes by attacking them using the mouse left button.
6. Collect all five coloured gems hidden across the run for a score bonus.
7. Survive as long as possible; speed increases progressively over time.

5 Rendering Pipeline

5.1 Scene Setup

The rendering context is managed by `main.js`, which initializes a `THREE.js` **Scene** and a **WebGLRenderer**. The renderer is configured to cover the full viewport, with antialiasing enabled and the power preference set to ‘high-performance’.

5.2 Textures and Materials

3D models loaded into the scene retain their embedded material properties. The chosen textures for the side and road are loaded depending on the chosen map. These textures are loaded via `THREE.js` **TextureLoader** and rely on `THREE.RepeatWrapping` to seamlessly tile across large ground planes without visible stretching or distortion. Shared materials, such as `MeshStandardMaterial`, are cached and applied to objects like the Standard Box to avoid redundant GPU uploads across multiple instances.

5.3 Lighting

The in-game scene utilizes two kinds of lighting:

- **Ambient light:** A uniform, omnipresent light source that illuminates all objects in the scene equally from every direction. The light color is white with 80% of light intensity.
- **Directional light:** A primary directional source with a 120% intensity positioned dynamically to simulate sunlight. It handles shading, highlights, and can cast shadows.

5.4 Camera

The viewport is driven by a `PerspectiveCamera` coupled with `OrbitControls` to smooth out manual tracking via movement damping. During gameplay, the camera dynamically anchors to the active character, maintaining an offset so that the relative perspective remains constant as the player traverses the map.

6 Technical Aspects

6.1 Asset Loading

The `main.js` orchestrates asset loading through standard JavaScript asynchronous Promises. A `Promise.all()` block is utilized to concurrently fetch all required files before gameplay begins:

- `GLTFLoader` — parses the ‘glb’ and ‘gltf’ hierarchies for Crash Bandicoot, Wumpa fruits, gems, and environmental boundaries.
- `FBXLoader` — handles the ‘fbx’ structure specifically required for Dr. Neo Cortex.

This promise-based pipeline ensures that all resources—including textures and geometries—are fully loaded into memory before the ‘`waitForPlay()`’ execution block resolves and initializes the scene.

6.2 Main Game Loop

The core of the game is the `animate()` function called each frame via `requestAnimationFrame`. If the game is not paused, each iteration:

1. Updates all active Tween animations, controlling object spin, and smooth transitions.
2. Checks intersections between the player’s bounding geometries and the hitboxes of all registered collectables and hazards.
3. Rewards points, extra lives, or deals damage based on the collected collision data.
4. Calculates the composite score and syncs it with the HTML HUD overlay.
5. Updates the camera’s coordinates to track the player’s continuously moving mesh.
6. Renders the final scene.

6.3 Map Generation

Infinite terrain generation is managed by `map_generation.js`. The map is built in modular chunks called “tiles” driven by a procedural spawn matrix:

1. A logical grid (rows and columns) determines the potential spawn points across the road's three lanes and the decorative sidelines.
2. Probabilistic selection logic populates these cells, maintaining a carefully balanced hazard density (scaling with difficulty) while ensuring safe paths exist.
3. A dedicated streak generator evaluates streaks of Wumpa fruits, dynamically switching lanes mid-streak to guide the player's movements.

6.4 Object Pooling and Memory Management

Because continuously parsing 3D files during gameplay causes severe lag, `SkeletonUtils.clone()` is used to duplicate pre-loaded model trees into active scene objects. A garbage-collection routine (`removeTiles`) tracks tiles that have fallen far behind the player's current position. Once a tile passes the removal threshold, it is deleted, its collision targets are unregistered, and a replacement tile is spawned ahead to maintain the endless runway.

6.5 Collision Detection

Collision detection executes every frame inside `check_collisions.js`. For optimal performance and accuracy, the system employs two different intersection strategies depending on the object type:

- **Axis-Aligned Bounding Box (AABB):** Used for crates, Wumpa fruits, gems, and dropped lives. `THREE.Box3` boundaries are evaluated against the character's hitbox. An intersection is registered when geometries overlap across all three axes simultaneously.
- **Bounding Spheres:** Used specifically for gears. Because gears rotate rapidly and are circular, `intersectsSphere` is used with a radial hitbox of 3.5 units. This provides a much fairer hitbox for the player when dodging diagonal edges.

For debugging, the game includes a visual hitbox toggle which draws wireframe representations directly into the scene. This toggle is present in the pause menu of the game.

6.6 Pause System

Pressing `Escape` calls `pauseGame()` in `game_management.js`. This function:

- Displays the pause HTML overlay (`pause.html`).
- Invokes `pauseAllTweens()` to freeze animations exactly where they are.
- Pauses the execution of core logic inside the `animate()` loop via the `isPaused` boolean flag.

6.7 Settings System

`settings.js` maintains a global `Settings` class initialized with parameters selected by the player on the main menu:

- `character` — Crash or Cortex.
- `difficulty` — Dictates base speed, maximum speed, acceleration rates, starting lives, and spawn density.
- `map` — Defines the specific environmental textures and 3D boundary assets loaded into the background.

The settings object dynamically calculates forward speed at runtime based on the total distance the player has covered, capping it at a maximum value to ensure the game remains beatable.

7 Animations

All animations are authored directly in JavaScript using Tween.js. No pre-baked clips are imported; every motion is defined by a chain of Tween objects applied to bone rotations, translations, and mesh coordinates.

7.1 Character Animations

7.1.1 *Forward Movement*

A scheduled, looping tween sequence translates the character forward along the X-axis in linear 500-unit segments, calculating duration dynamically based on the current game speed.

7.1.2 *Run*

A cyclic locomotion animation that utilizes continuous sine wave tweens. For Crash, the lower-body cycle drives leg swings, knee flexes, foot rocks, toe flicks, and a double-frequency vertical body bounce. The upper body is synchronized with a secondary tween that swings the arms contra-laterally to the legs, bending the elbows acutely on the forward swing and straightening them on the backswing. For Cortex, the animation applies sinusoidal offsets from his saved canonical T-pose, bringing the arms down to a natural idle while applying leg amplitudes and a subtle spinal twist.

7.1.3 *Jump Animation*

Triggered to clear obstacles, preventing double-jumping through a boolean check. The character's mesh is translated upward 6 units on the Y-axis using quadratic out easing, then falls back to the ground plane using quadratic in easing before unlocking the jump state.

7.1.4 *Lateral Slide Animation*

Triggered to smoothly transition the character laterally between the three lanes. The character tweens across the Z-axis over 250 milliseconds. Utilizing a calculated progress ratio, the animation applies a temporary sine-wave lean angle (Z-rotation) that simulates leaning into the turn before returning perfectly upright.

7.1.5 *Crash Spin Attack*

A rapid, three-phase procedural animation: a 50-millisecond anticipation phase where the knees bend and the arms snap into a T-pose; a fast 400-millisecond spin phase consisting of three full rotations around the Y-axis with a sharp, explosive custom easing curve; and a 50-millisecond recovery phase easing back to the neutral walk cycle.

7.1.6 *Cortex Punch Attack*

A sequential three-phase strike utilizing Cortex's specific bone rig. It begins with a 100-millisecond wind-up that pulls the right arm back, flexes the elbow, and twists the spine. This is followed by an aggressive 100-millisecond forward lunge and punch, before finally executing a 200-millisecond recovery tween that smoothly blends all bones back into the running pose.

7.2 Object Animations

To manage memory and allow for global pausing, all object animations are tracked via a `registerObjectTween` helper function, which pushes active tweens to the object's `userData` array.

7.2.1 *Wumpa Fruits*

To draw the player's attention, the fruits are assigned a continuous, looping tween that executes a full 360-degree rotation around the Y-axis every 3000 milliseconds. Concurrently, chained tweens create a gentle 0.6-unit bobbing motion up and down.

7.2.2 *Nitro Boxes*

Nitro boxes perform a sequential idle loop to stand out: an upward hop, landing, rotating 30 degrees to the right, returning to center, rotating 30 degrees to the left, and returning to center again before using an `onComplete` callback to repeat the cycle indefinitely.

7.2.3 *Dropped Items*

When standard or question boxes are broken, the dropped item triggers an initial quick upward pop over 300 milliseconds to simulate bursting out of the crate, before settling back down to its base height. Once settled, it inherits the standard Wumpa-style continuous spin.

7.2.4 *Aku Aku Startup*

Upon collection, the Aku Aku mask executes an introductory sequence: a brief 200-millisecond wait, a 25-degree tilt to the left, a return to neutral, a 25-degree tilt to the right, a return to neutral, and finally a smooth 180-degree spin to face and greet the player.

7.2.5 *Gears (Hazards)*

Metallic gears are constantly moves in the map. A perpetual translation tween slides them 8.5 units left and right across the Z-axis lanes over 4000 milliseconds, while a simultaneous rotation tween spins them rapidly around their Y-axis.

8 Audio System

Audio is managed by `sounds.js` utilizing native HTML5 `Audio` objects rather than external 3D engine components. All sounds operate globally as non-positional sources, meaning their volume remains uniform regardless of the player's in-world coordinates.

Instead of rigid hardware-style audio channels, the system segregates audio management into two distinct categories using standard web APIs:

- **Background Music:** Controlled via a dedicated global audio instance (`mainThemeAudio`) that handles looping the main soundtrack. It can be independently toggled, paused, or reset via specific export functions.
- **Sound Effects:** Managed dynamically through a custom `Sound` class wrapper. Every instantiated sound effect automatically registers itself into a global array (`sfxInstances`).

9 File Structure and Organisation

public/ All static assets served by Vite:

models/ GLTF, GLB, and FBX 3D model files.

textures/ Tile and environment texture images.

sounds/ Music tracks and sound effect files.

main.js Entry point. Initialises the `THREE.js` scene, renderer, camera, and lights; coordinates the `LoadingManager`; starts the render loop.

settings.js Global settings and runtime state object. Defines difficulty presets and map configuration data.

characters.js `Crash` and `Cortex` class definitions. Each class stores references to the loaded model and bone handles used by the animation system.

character_management.js Initialises the active character, sets bone references, and transitions between home-page and in-game configurations.

character_animations.js All character animations: idle, follow-mouse, run, jump, slide, attack, damage, star, lane-change, and reset.

map_generation.js Tile creation, spawn matrix construction, object placement, tile removal, and tween cleanup for off-screen tiles.

objects.js Class definitions for all game objects (boxes, fruits, gems, gears) with scale, rotation, and position parameters.

objects_animations.js Tween-based animations for all non-character objects.

check_collisions.js AABB collision detection loop called each frame.

game_management.js Pause/resume, game-over, menu transitions, and HUD updates.

sounds.js Audio channel initialisation, music/sound play functions.

listeners.js *Tracks pressed key state (WASD, left mouse button) for input polling*

tween_registry.js Global list of all active Tweens; utility functions to pause, resume, and stop them collectively.

index.html Home page with character selection, difficulty, map, and start button.

hud.html / hud.css In-game heads-up display overlay.

pause.html / pause.css Pause menu overlay.

gameover.html Game over screen with final stats.
style.css Global stylesheet for menus and page layout.
vite.config.js Vite build configuration (base path, output directory).
package.json Project manifest listing THREE.js and Tween.js as dependencies.

10 Conclusions

N. Sane Runner is an effective example of a complete, browser-based interactive graphics project. It combines real-time 3D rendering via THREE.js and WebGL with procedural infinite-terrain generation, frame-accurate AABB collision detection, fully procedural character and object animations driven by Tween.js, and a multi-channel audio system—all delivered without a game engine or a UI framework.

Several aspects of the implementation merit particular attention. The tile generation and recycling system keeps memory usage bounded regardless of run length. Furthermore, defining every motion as a chain of tweens rather than importing pre-baked clips, provides fine-grained control on the complex hierarchical models which are present in this project.

In conclusion, N.Sane Runner demonstrates how complex, real-time 3D interactive applications can be built effectively within the browser framework using only low-level libraries.